# 在线判题系统（OJ） - 项目架构以及任务分配

## 一、项目架构设计

### 1. 系统架构概述

本次OJ系统采用前后端分离架构，后端使用C++实现，前端使用Vue.js框架，数据库使用MySQL，评测系统使用C++进程隔离实现沙箱环境。

### 2. 核心模块架构

#### 2.1 后端架构（C++）

- **用户管理模块**：负责用户认证、个人信息管理、权限控制
- **题目管理模块**：负责题目发布、编辑、删除、分类与搜索
- **代码评测模块**：负责代码提交处理、编译运行、结果判定
- **排行榜模块**：负责用户排名计算、题目统计、比赛管理
- **讨论区模块**：负责题目讨论、评论回复、内容管理
- **公共服务模块**：负责数据库操作、缓存管理、日志记录等基础设施
- **HTTP服务器**：使用C++ Web框架（如Crow/Httplib）实现RESTful API

#### 2.2 前端架构

- **页面组件**：首页、用户中心、题目列表、题目详情、代码编辑器、讨论区等
- **状态管理**：使用Vuex管理全局状态
- **路由管理**：使用Vue Router实现页面导航
- **API交互**：使用Axios与后端API通信

#### 2.3 评测系统架构

- **提交队列**：使用C++消息队列管理评测任务
- **沙箱环境**：使用进程隔离实现安全的代码运行环境
- **多语言支持**：支持C、C++、Java、Python等多种编程语言
- **评测引擎**：C++实现，执行测试用例、比对结果、计算运行时间和内存使用

### 3. 技术栈选择

| 类别 | 技术 | 版本 | 用途 |
|---|---|---|---|
| 后端语言 | C++ | C++17 | 核心业务逻辑实现 |
| 后端框架 | Crow/Httplib | Latest | HTTP服务器和RESTful API |
| 前端框架 | Vue.js | 3.x | 构建响应式前端界面 |
| 数据库 | MySQL | 8.0 | 存储用户、题目、提交等数据 |
| 缓存 | Redis | 7.0+ | 缓存热点数据、排行榜 |

| 类别 | 技术 | 版本 | 用途 |
|---|---|---|---|
| 消息队列 | RabbitMQ | 3.9+ | 管理评测任务队列 |
| 容器化 | Docker | 20.10+ | 构建沙箱环境（Java/Python评测） |
| 认证 | JWT | - | 实现无状态认证 |
| JSON处理 | nlohmann/json | Latest | JSON序列化和反序列化 |
| 数据库连接 | MySQL Connector/C++ | 8.0 | C++数据库操作 |

# 二、任务分配

## 1. 团队成员及分工

| 成员 | 负责模块 | 主要任务 | 技术栈 |
|---|---|---|---|
| 成员1（Java+C++） | 代码评测模块 | 1. 代码提交接口<br>2. 编译与运行环境搭建<br>3. 沙箱安全机制<br>4. 评测结果判定与反馈<br>5. 多语言支持扩展 | C++、Docker、进程隔离 |
| 成员2（Java+C++） | 题目管理模块 | 1. 题目发布、编辑、删除<br>2. 题目分类与标签管理<br>3. 题目搜索与筛选<br>4. 测试用例管理<br>5. 题目数据接口 | C++、MySQL、JSON |
| 成员3（C++） | 用户管理模块 | 1. 用户注册、登录、登出功能<br>2. 个人信息管理<br>3. 权限管理与控制<br>4. JWT认证实现<br>5. 用户数据接口 | C++、MySQL、JWT |
| 成员4（C++） | 公共服务模块 | 1. HTTP服务器框架搭建<br>2. 数据库连接池实现<br>3. 日志系统实现<br>4. 缓存管理（Redis）<br>5. 全局异常处理 | C++、MySQL、Redis |
| 成员5（C++） | 排行榜模块 | 1. 总排行榜实现<br>2. 题目完成情况统计<br>3. 比赛排行榜（可选）<br>4. 排名算法优化<br>5. 排行榜数据接口 | C++、Redis、MySQL |

| 成员 | 负责模块 | 主要任务 | 技术栈 |
|---|---|---|---|
| 成员6（C++） | 讨论区模块 | 1. 讨论主题管理<br>2. 评论与回复功能<br>3. 讨论区管理与审核<br>4. 内容安全过滤<br>5. 讨论区数据接口 | C++、MySQL |

**备选方案**：成员6可调整为负责前端界面模块（Vue.js），前端任务相对独立，2位Java成员可协助前端开发。

### 2. 关键技术点与挑战

| 模块 | 关键技术点 | 挑战 | 解决方案 |
|---|---|---|---|
| 代码评测 | 沙箱安全、多语言支持 | 安全性与性能平衡 | 使用Docker容器隔离，进程资源限制，多线程处理 |
| 题目管理 | 题目搜索、测试用例管理 | 数据量大时的性能 | 使用MySQL索引优化，测试用例存储优化 |
| 用户管理 | JWT认证、密码加密 | 安全性 | 使用bcrypt加密密码，JWT设置合理过期时间 |
| 公共服务 | 数据库连接池、高并发 | 性能与稳定性 | 使用连接池复用，异步日志记录 |
| 排行榜 | 实时排名、数据更新 | 高并发下的性能 | 使用Redis缓存，定时更新与实时计算结合 |
| 讨论区 | 内容审核、评论管理 | 内容安全 | 实现关键词过滤，管理员审核机制 |

### 3. 技术储备建议

由于团队中有4位成员只会C++，建议：

- **C++ Web框架学习**：Crow、Restbed、Httplib等
- **C++数据库操作**：MySQL Connector/C++、soci、DAO等
- **C++ JSON处理**：nlohmann/json、rapidjson等
- **C++ Redis客户端**：hiredis、redis-plus-plus等
- **前端学习**：Vue.js入门简单，2位Java成员可快速上手协助

## 三、项目管理与协作

### 1. 版本控制

- 使用Git进行版本控制
- 分支管理策略：main（主分支）、dev（开发分支）、feature/xxx（功能分支）

### 2. 代码规范

- 后端：遵循Google C++代码规范，使用C++17特性
- 前端：遵循Vue.js代码规范，使用ESLint进行代码检查

### 3. 文档管理

- API文档：使用OpenAPI/Swagger规范编写
- 技术文档：使用Markdown编写，包含架构设计、接口说明等
- 部署文档：包含环境配置、部署步骤等

### 4. 沟通与协作

- 每日站会：同步进度，解决问题
- 代码评审：确保代码质量
- 问题跟踪：使用Issue管理工具跟踪问题

通过以上架构设计和任务分配，结合团队成员的技术栈情况（2位Java+C++，4位C++），确保项目可以高效、有序地进行开发，同时保证系统的可扩展性和可维护性。